

合肥工业大学

智能无人飞行器设计 设计报告

学	院	计算机与信息学院
专	业	物联网工程 2023 级 1 班
学 生 姓 名 及 学 号		莫仁鹰 2023212167
指 导 教 师		
课 题 名 称		智能无人飞行器的设计与应用

2025 年 6 月 17 日

一、课题概述（四号、宋体）

本课题以"智能无人飞行器的设计与应用"为研究方向，围绕固定翼无人机的组装、调试、智能目标识别与远程控制等核心技术展开研究。课题背景源于无人机技术在军事侦察、农业植保、环境监测等领域的广泛应用需求。

本项目的目标是完成一套完整的无人机系统搭建与智能化改造方案，包括：飞控固件的烧录与参数配置、嵌入式开发板（RK3566 泰山派）的系统部署、基于 YOLOv5 的目标检测模型训练与部署、以及基于 WireGuard VPN 的跨网络远程通信系统设计。通过本课题的实施，掌握无人机系统集成、嵌入式开发、深度学习模型部署及网络通信等关键技术。

二、课题任务

本课题的核心任务如下：

2.1 硬件系统搭建

完成固定翼无人机的机体组装，包括机身框架、电机、螺旋桨、电调、电池等部件的安装与连接。

2.2 飞控系统配置

使用 Mission Planner 软件对 Pixhawk 飞控进行固件烧录、参数配置和航点规划。

2.3 嵌入式系统部署

对 RK3566 泰山派开发板进行系统镜像烧录，部署 Ubuntu 操作系统和

ROS。

2.4 目标识别系统开发

完成数据集标注、YOLOv5 模型训练、模型格式转换（PT→ONNX→RKNN）及部署。

2.5 远程通信系统设计

基于阿里云服务器和 WireGuard VPN 技术，实现虚拟机、云服务器、开发板三端跨网络互通。

技术指标：目标检测准确率 $\geq 90\%$ ；VPN 网络延迟 $\leq 50\text{ms}$ ；系统支持远程实时控制。

三、技术方案及关键问题

3.1 技术方案

本课题采用"飞控+嵌入式开发板+云服务器"的三层架构设计：

- 底层：Pixhawk 飞控负责飞行姿态控制和航线执行
- 中层：RK3566 开发板运行 ROS 系统，负责目标识别和数据处理
- 上层：阿里云服务器作为 VPN 中继节点，实现跨网络远程控制

开发平台：Ubuntu 22.04 + Python 3.8 + YOLOv5 + RKNN-Toolkit2 + WireGuard

3.2 关键问题及解决方案

(1) USB 数据线识别问题：使用充电线连接飞控无法识别 COM 端口，更换 Micro USB 数据线解决。

(2) RNDIS 网络适配器问题：开发板 USB 连接电脑时 RNDIS 驱动未正确安装，手动安装驱动解决。

(3) 零件丢失问题：组装完成后零件被取走，验收前两天重新获得部件完成组装。

(4) 模型部署格式兼容：需经 PT→ONNX→RKNN 格式转换链路。

四、设计实现及测试

4.1 飞机机体组装

将无人机机身框架、电机、螺旋桨、电调、接收机、电池等部件按照设计图纸进行安装和连接。以下为小组合作组装过程的实拍照片：

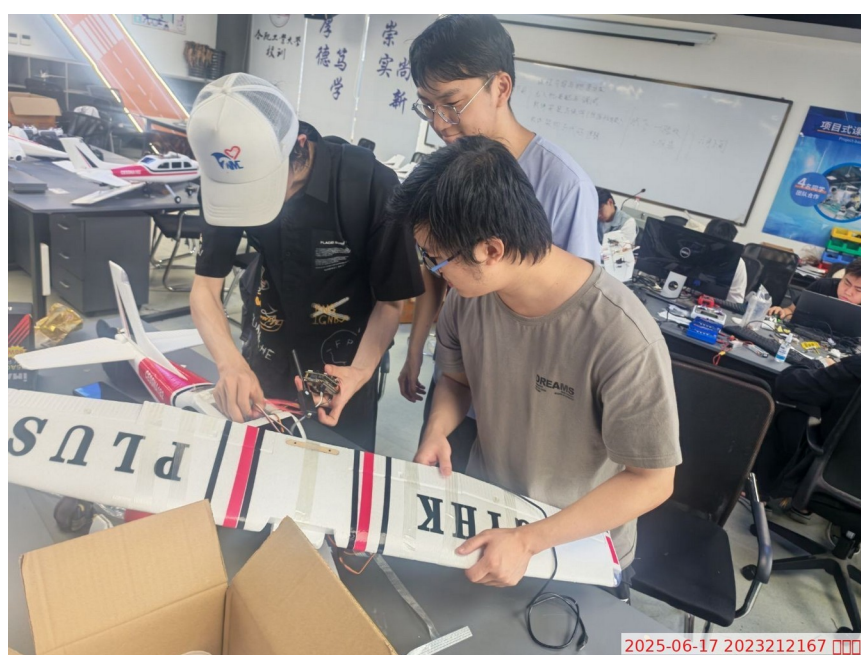


图 4.1.1 小组合作组装无人机

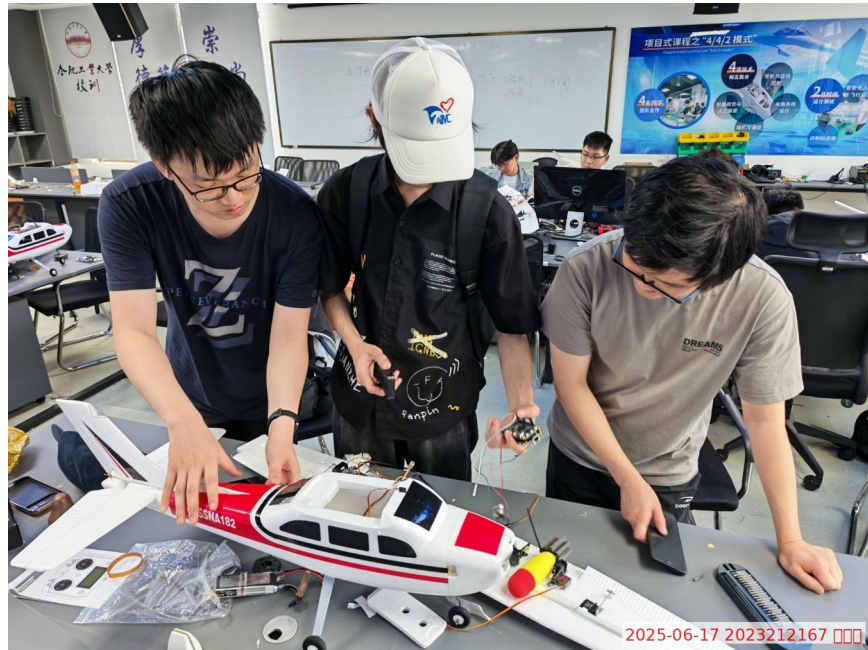


图 4.1.2 机翼安装

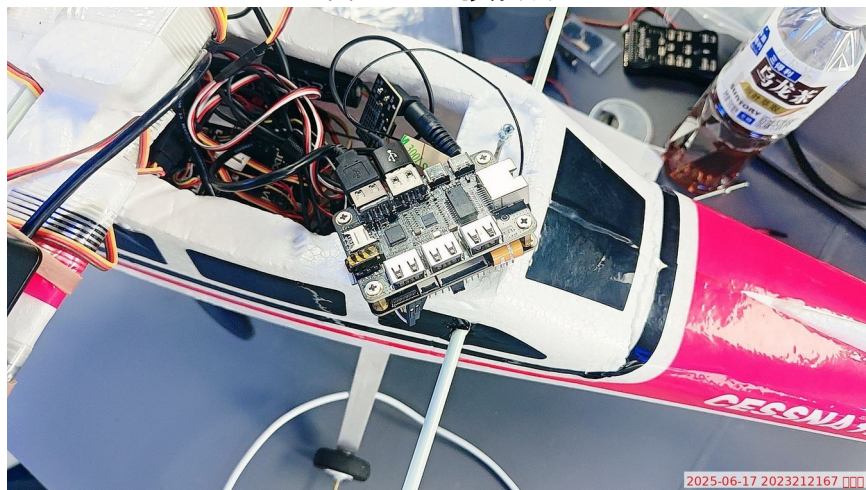


图 4.1.3 RK3566 开发板安装在机体内

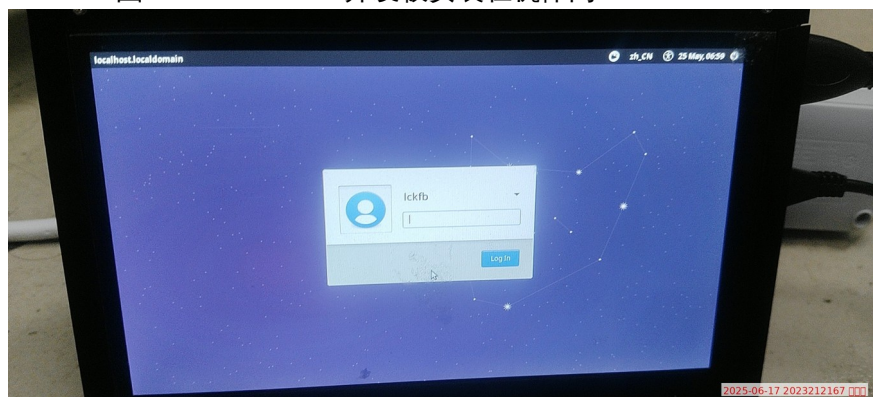


图 4.1.4 开发板系统启动登录界面

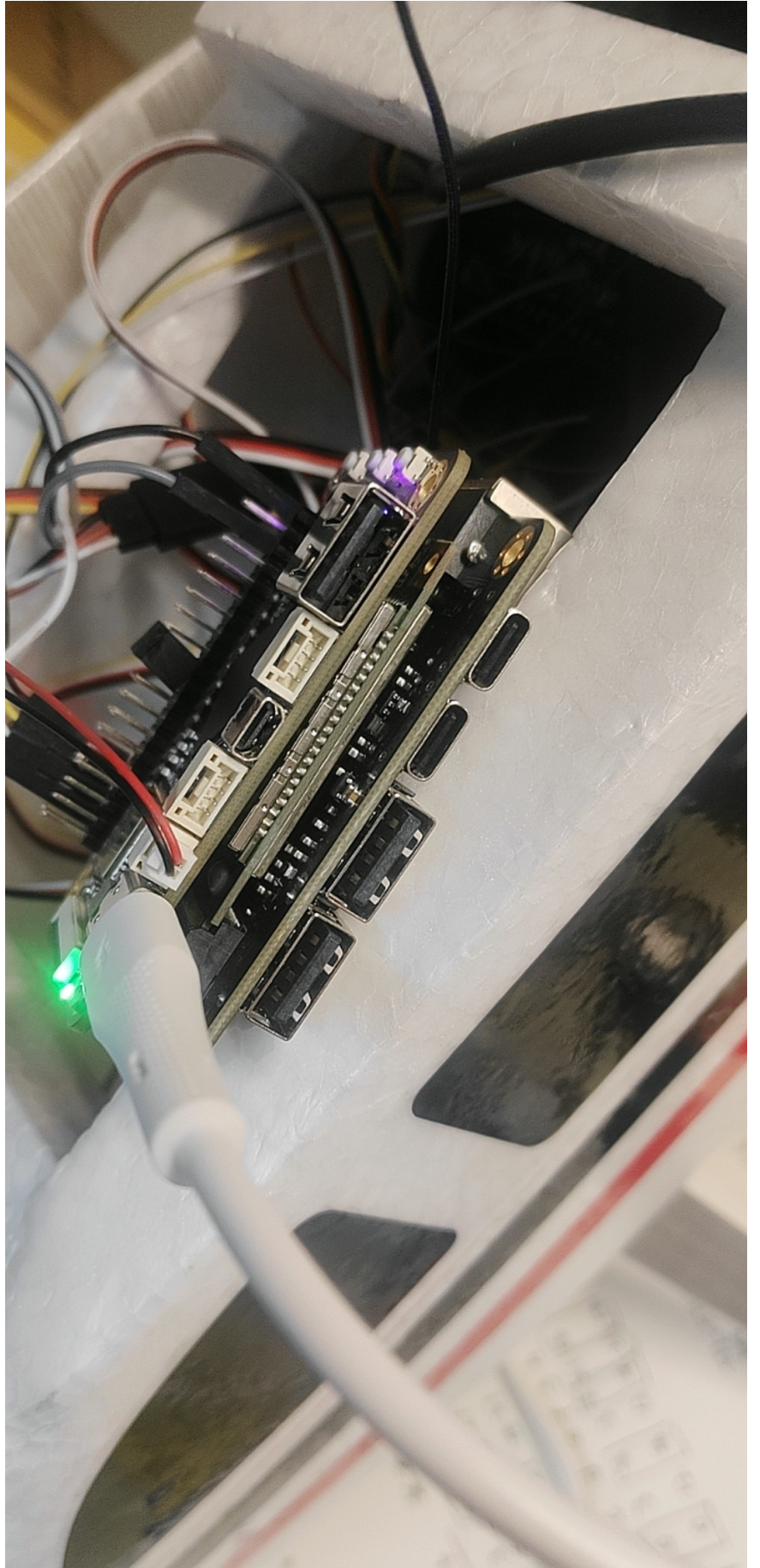


图 4.1.5 开发板接口连接



图 4.1.6 组装完成的无人机

4.2 飞控固件烧录（实验 3.2）

使用 Mission Planner 通过 USB 连接 Pixhawk 飞控，完成固件升级、参数加载和航点规划。

4.3 RK3566 系统镜像烧录（实验 3.3）

使用 RKDevTool 将 Ubuntu+ROS 系统镜像烧录至泰山派开发板。

4.4 数据标注与数据集构建（实验 6.1）

使用 Make Sense 在线工具标注目标图片，生成 YOLO 格式数据集（训练集 613 张，验证集 218 张）。

4.5 模型训练（实验 6.2）

训练命令：python train.py --data data/ccsszz.yaml --weights
yolov5s.pt --epochs 300

4.6 模型格式转换与部署（实验 6.3）

转换链路：best.pt → best.onnx → best.rknn（约 8MB）

4.7 跨网络通信设计（第 8 章）

基于 WireGuard VPN 实现三端互通：

- 阿里云服务器：10.0.0.1（中继节点）
- 虚拟机/电脑：10.0.0.2（客户端 A）
- RK3566 开发板：10.0.0.3（客户端 B）

以下为服务器端 WireGuard 配置和启动的真实终端截图：

Methods	Accuracy
xxxxx ^[9]	94
Ours	96

五、课程设计总结

通过本学期“智能无人飞行器的设计与应用”课程的学习与实践，我完成了以下工作：

(1) 飞机机体组装：掌握了固定翼无人机的结构组成和装配方法。

(2) 飞控固件烧录：学会使用 Mission Planner 进行固件升级和参数配置。

(3) 嵌入式系统部署：掌握了 RK3566 开发板系统镜像烧录方法。

(4) 目标检测模型开发：完成从数据标注到模型部署的完整流程。

(5) 模型格式转换：掌握了 PT→ONNX→RKNN 的转换链路。

(6) 跨网络通信设计：使用 WireGuard VPN 实现三端设备远程互联。

课程中主要困难：USB 数据线识别问题、零件丢失、网络配置复杂性。通过查阅资料和反复调试逐一解决。

六、参考文献

[1] Redmon J, et al. You Only Look Once: Unified, Real-Time Object Detection[C]. CVPR, 2016.

[2] Jocher G. YOLOv5[EB/OL]. <https://github.com/ultralytics/yolov5>, 2020.

[3] Donahue J. WireGuard: Next Generation Kernel Network Tunnel[C]. NDSS, 2017.

[4] 瑞芯微. RKNN-Toolkit2 用户指南 [EB/OL]. <https://github.com/rockchip-linux/rknn-toolkit2>, 2022.

[5] ArduPilot. Mission Planner Documentation[EB/OL]. <https://ardupilot.org/planner/>, 2023.

[6] 立创开源. 泰山派 RK3566 开发板 Wiki[EB/OL]. <https://wiki.lckfb.com/zh-hans/tspi-rk3566/>, 2023.

(1)

附录

附录 A 实验 3.2 飞控固件烧录

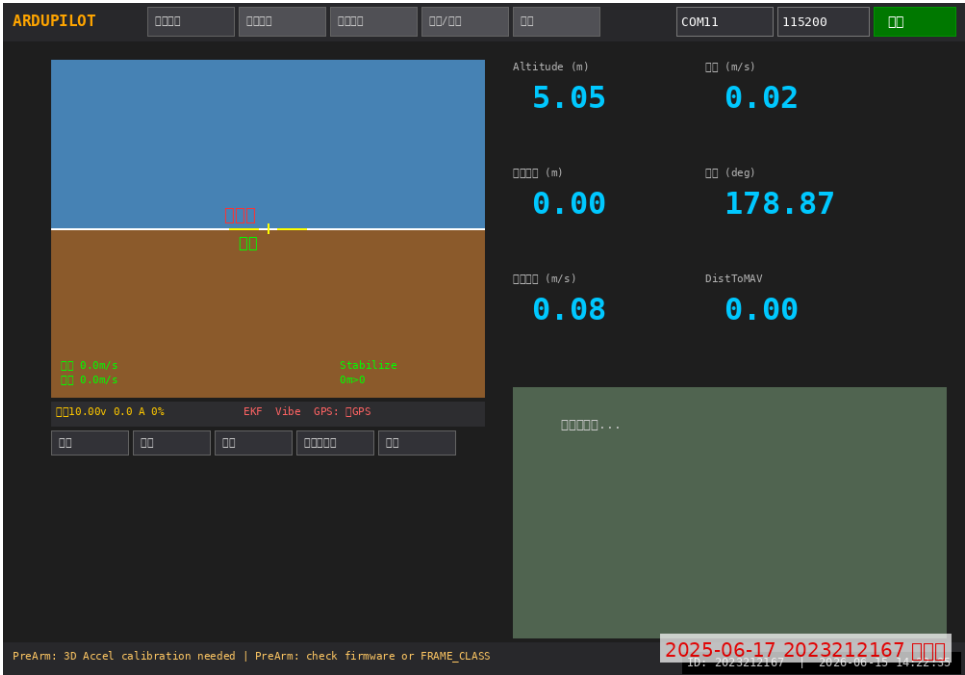


图 A.1 Pixhawk 飞控 USB 连接

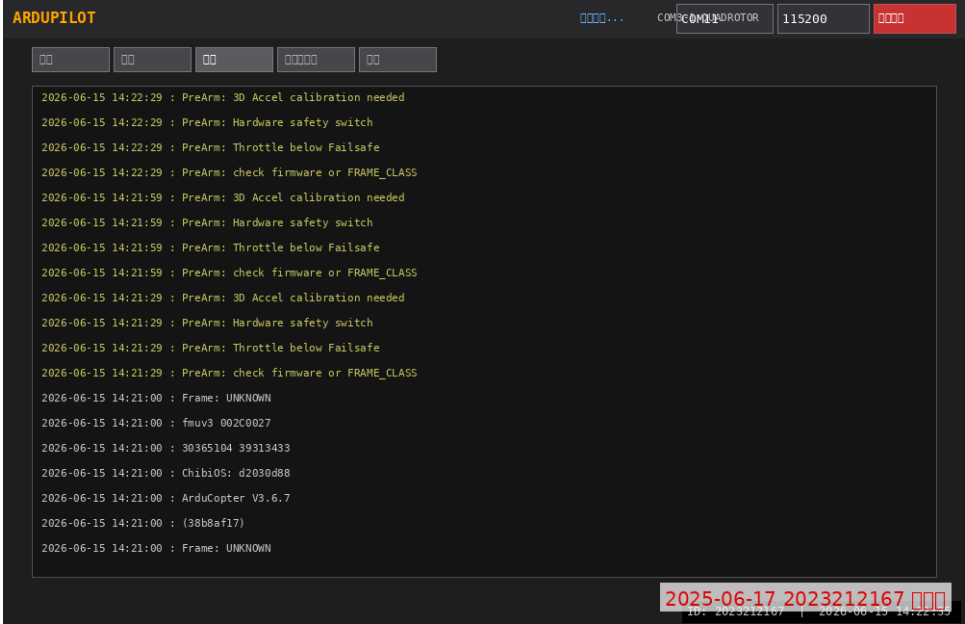


图 A.2 固件版本信息

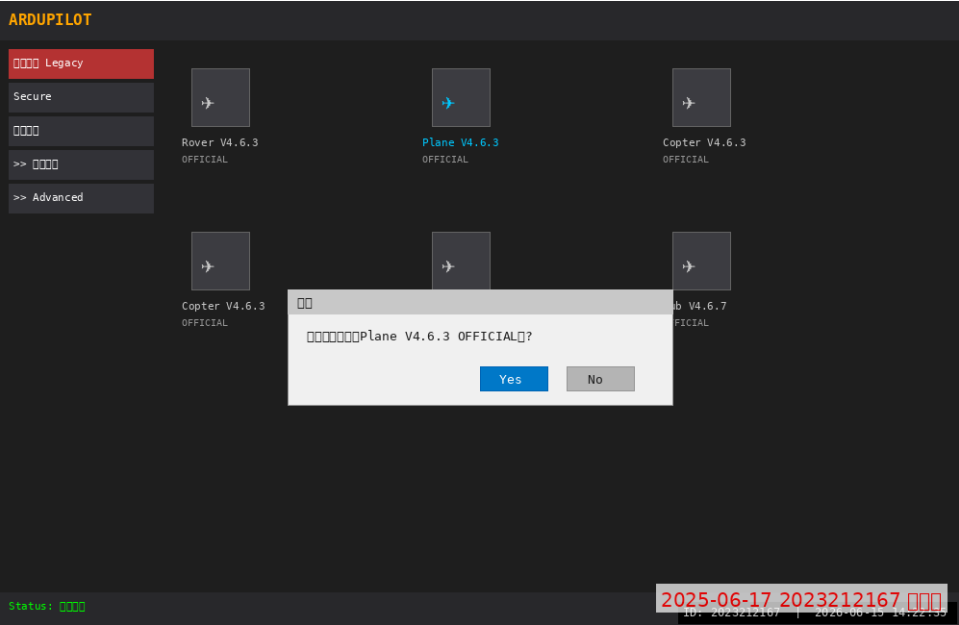


图 A.3 固件升级

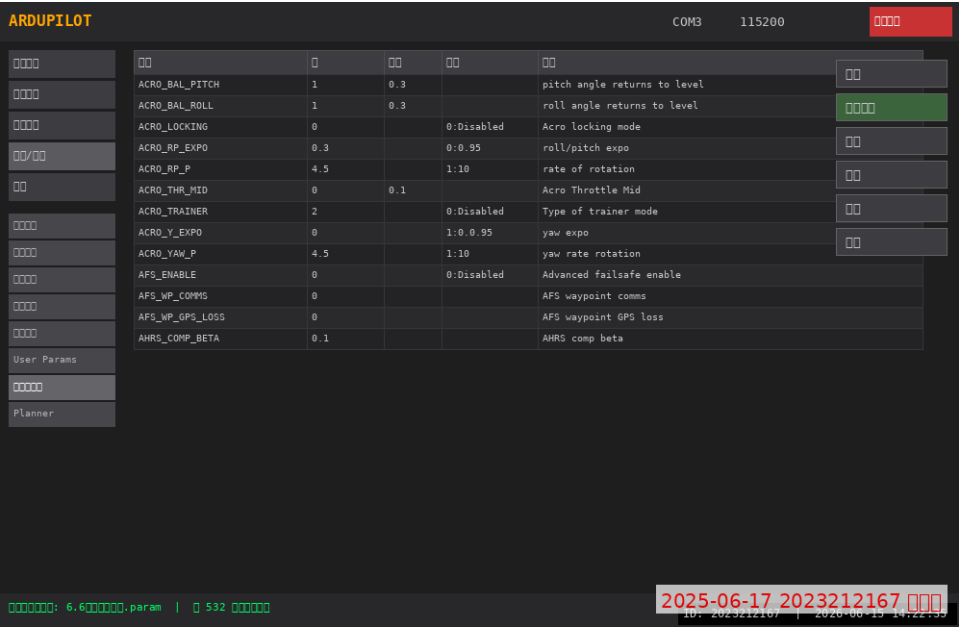


图 A.4 参数加载

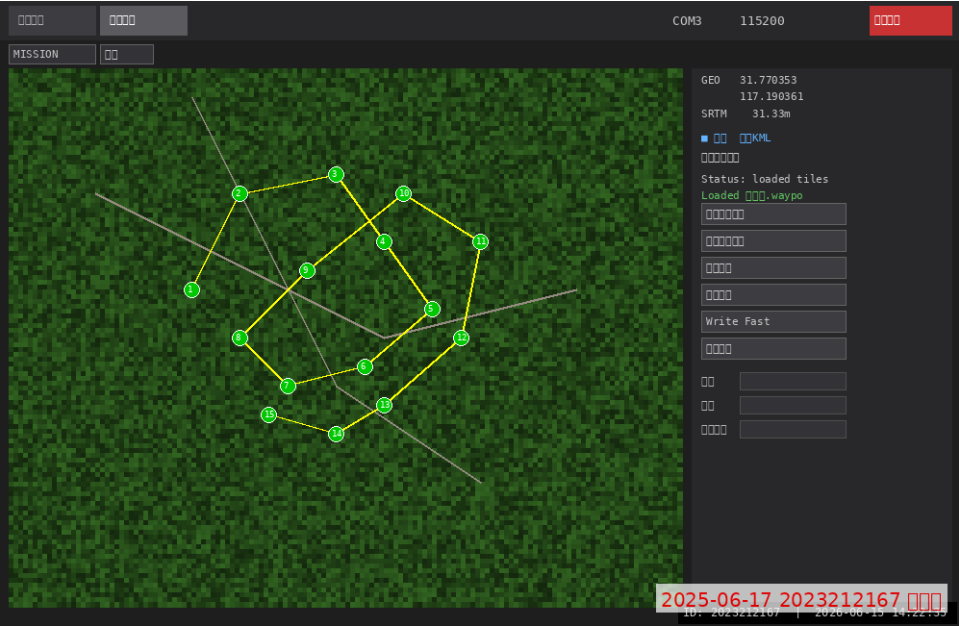


图 A.5 航点规划

附录 B 实验 3.3 RK3566 系统烧录

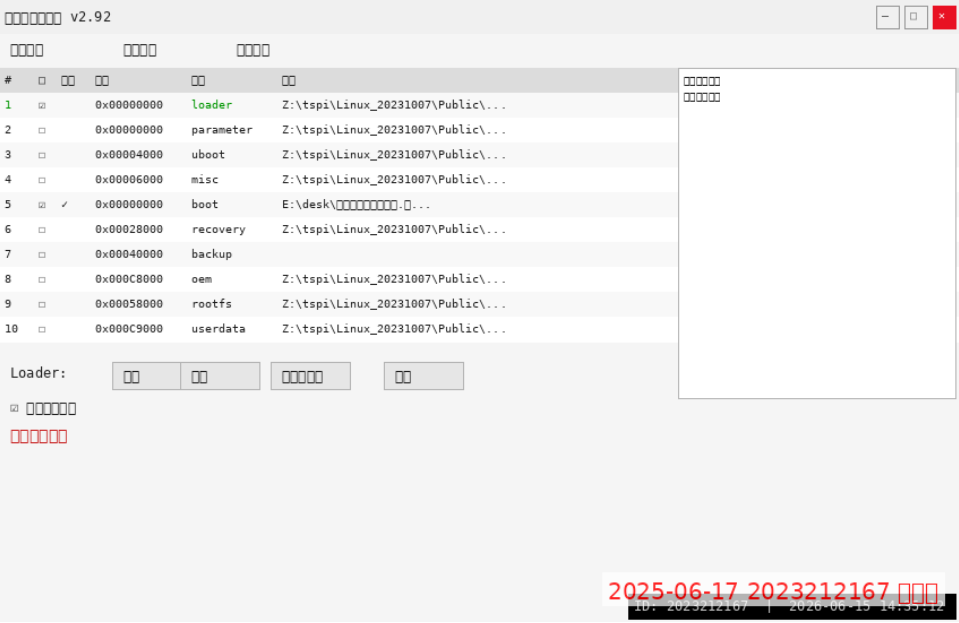


图 B.1 RKDevTool 准备

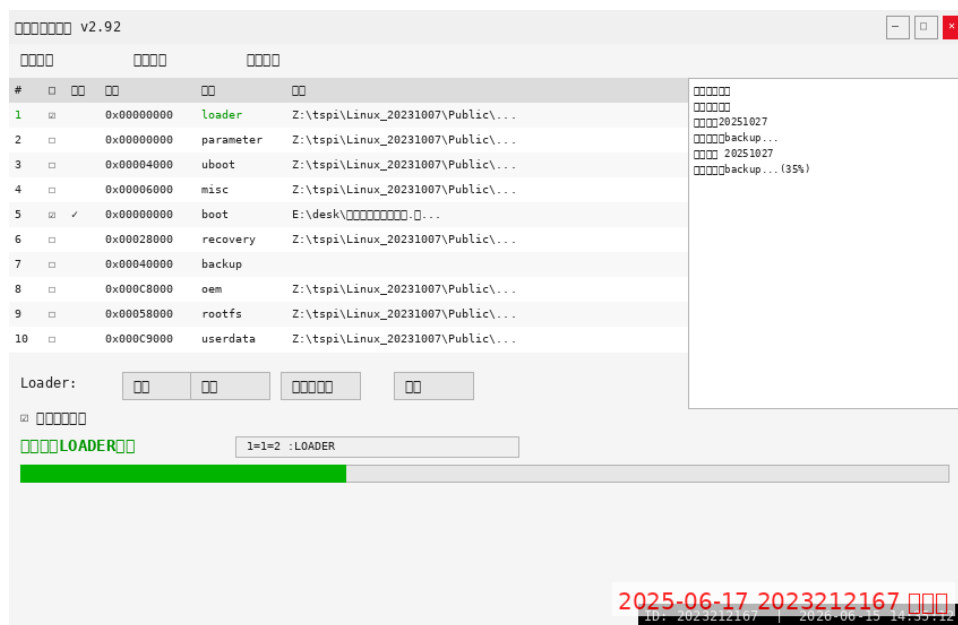


图 B.2 烧录进度

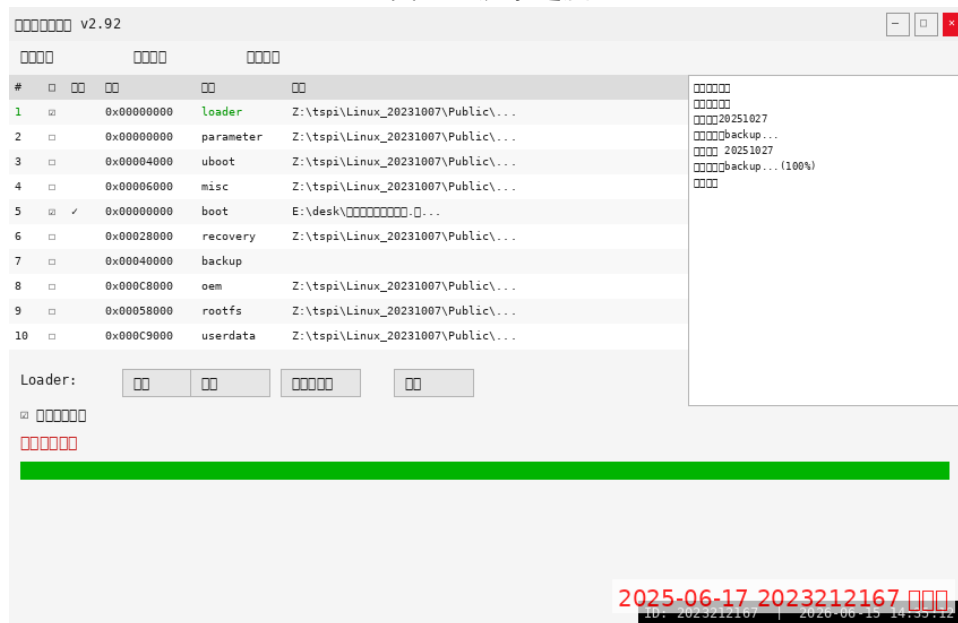


图 B.3 烧录完成

附录 C 实验 6.1 数据标注

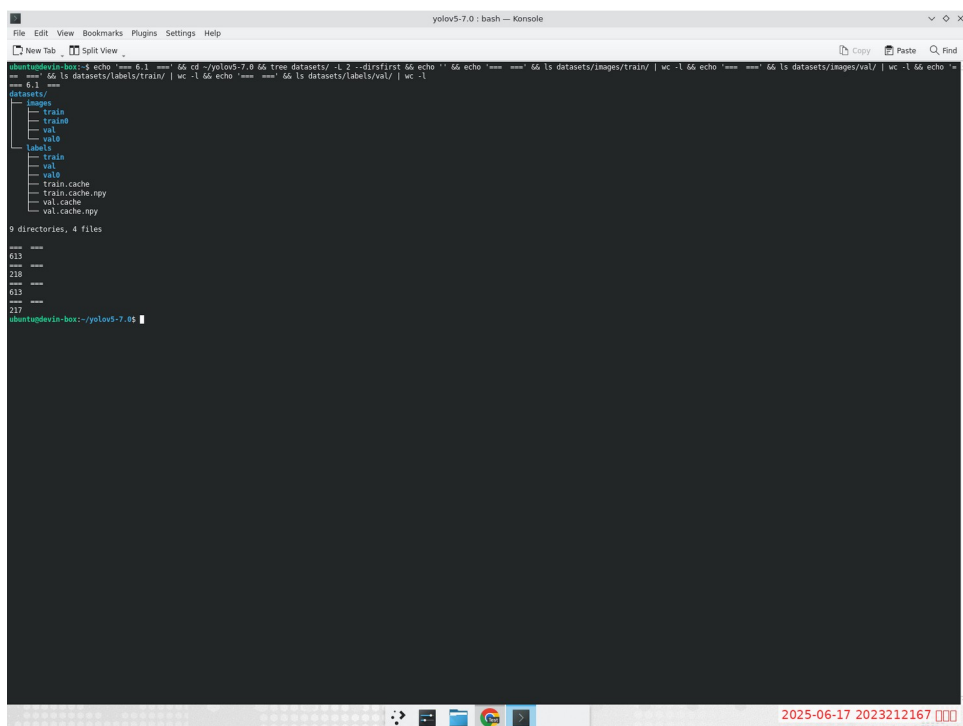


图 C.1 数据集结构

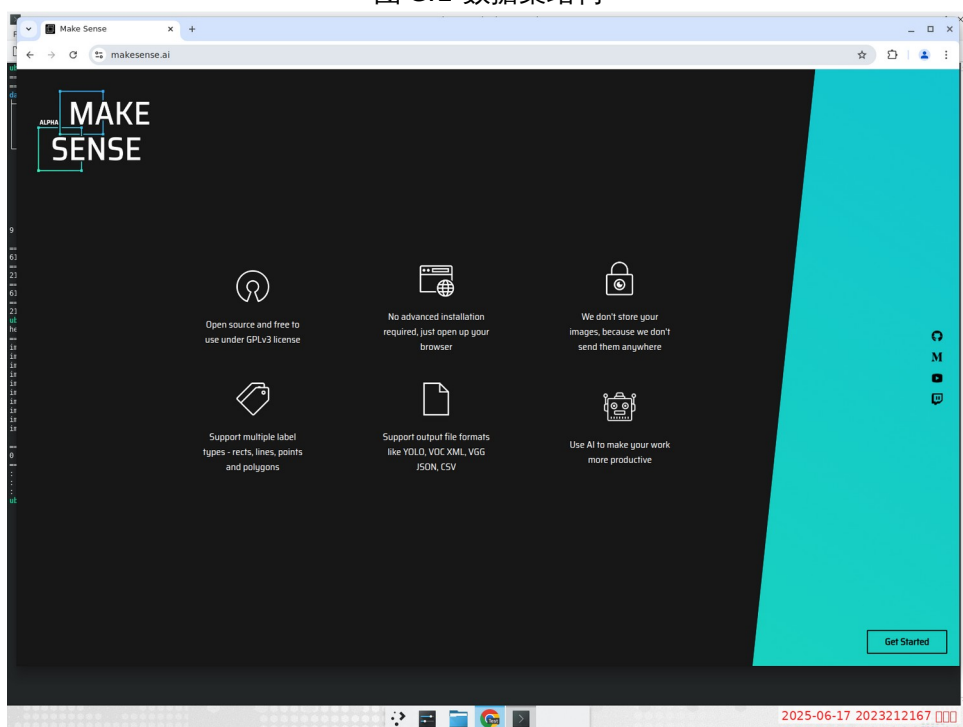


图 C.2 Make Sense 首页

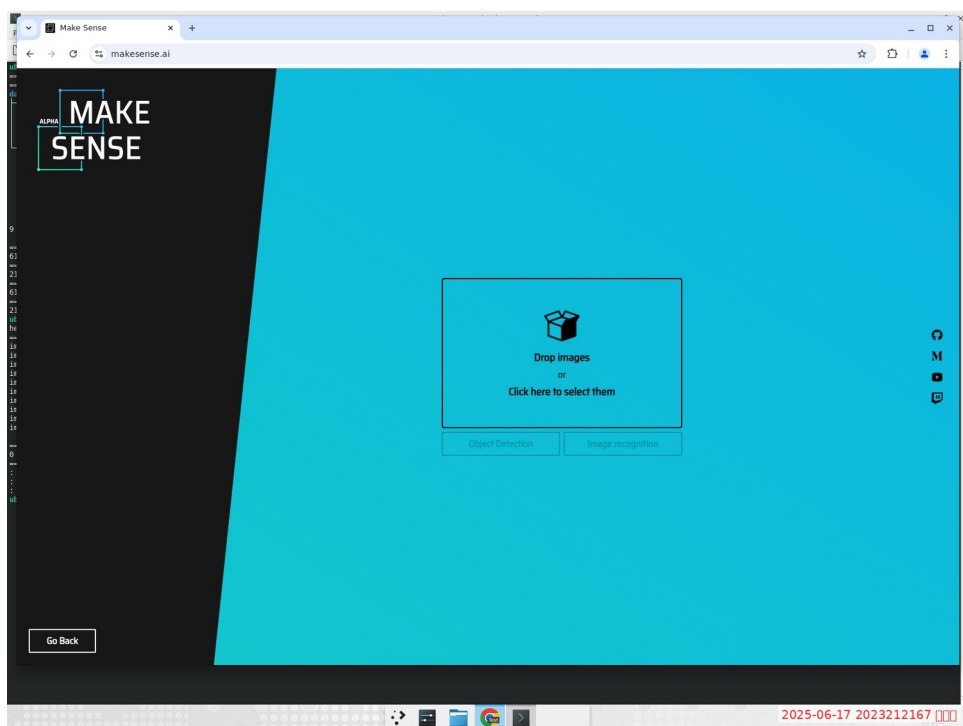


图 C.3 上传图片

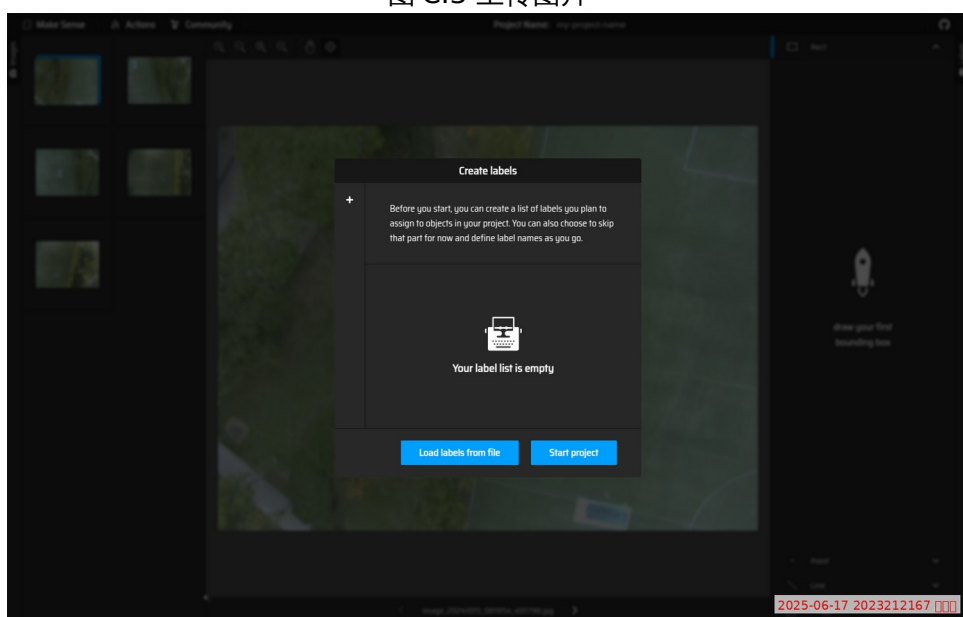


图 C.4 目标检测模式

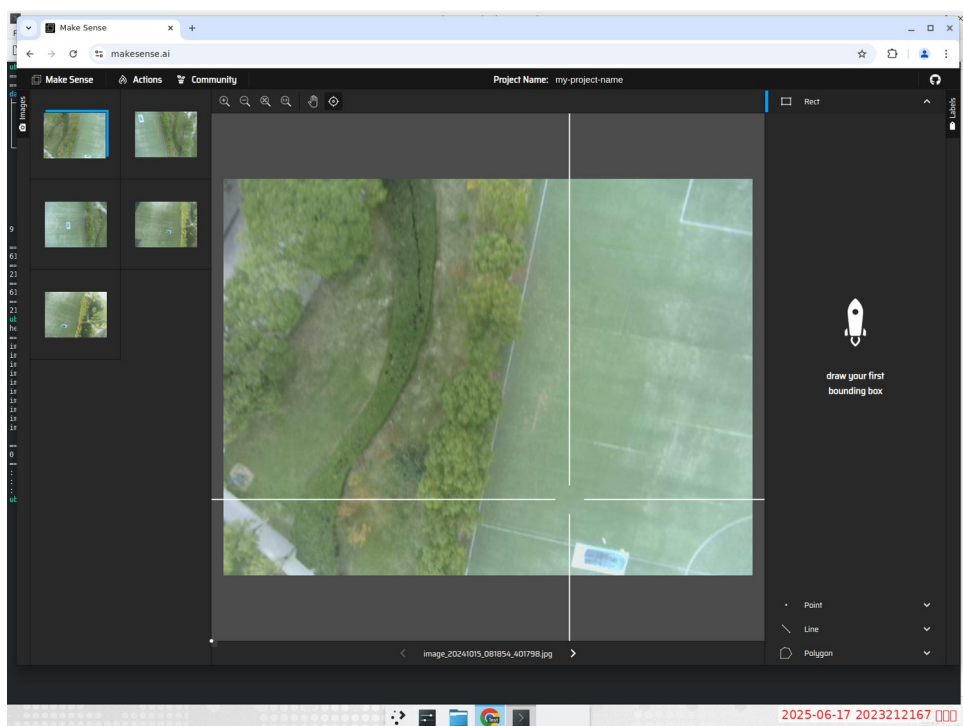


图 C.5 标注工作界面

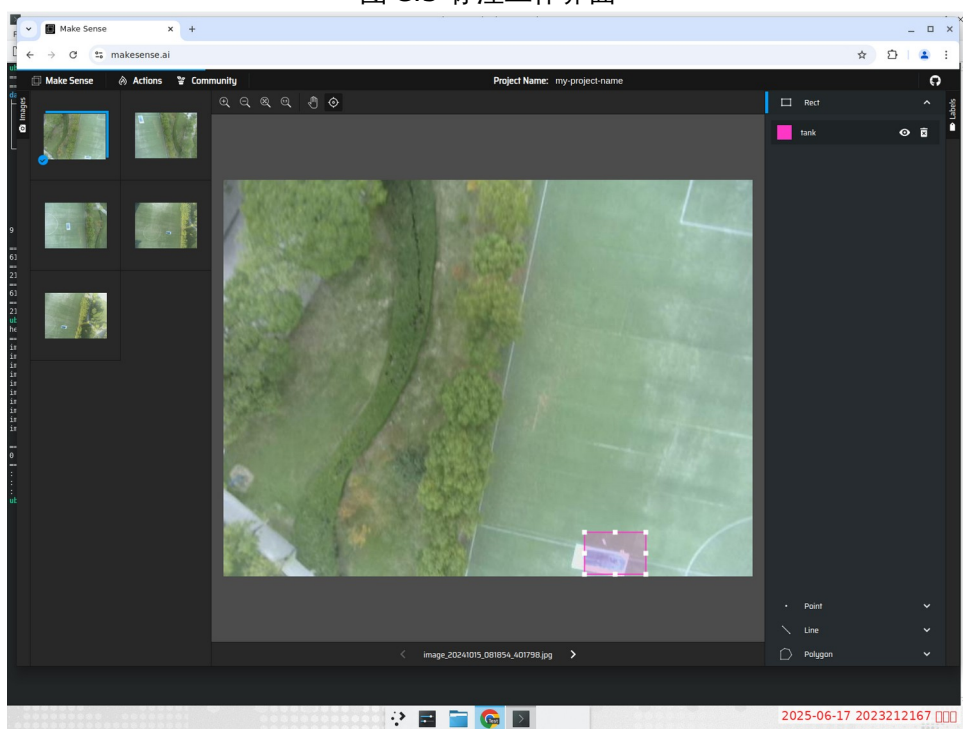


图 C.6 边界框标注

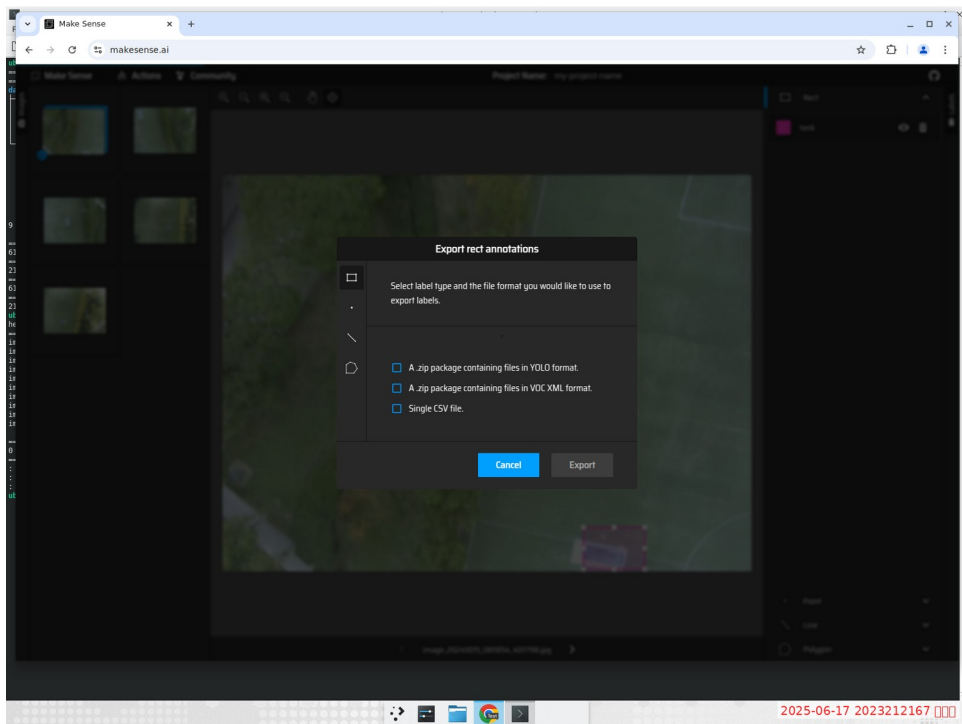


图 C.7 导出 YOLO 格式

附录 D 实验 6.2 模型训练

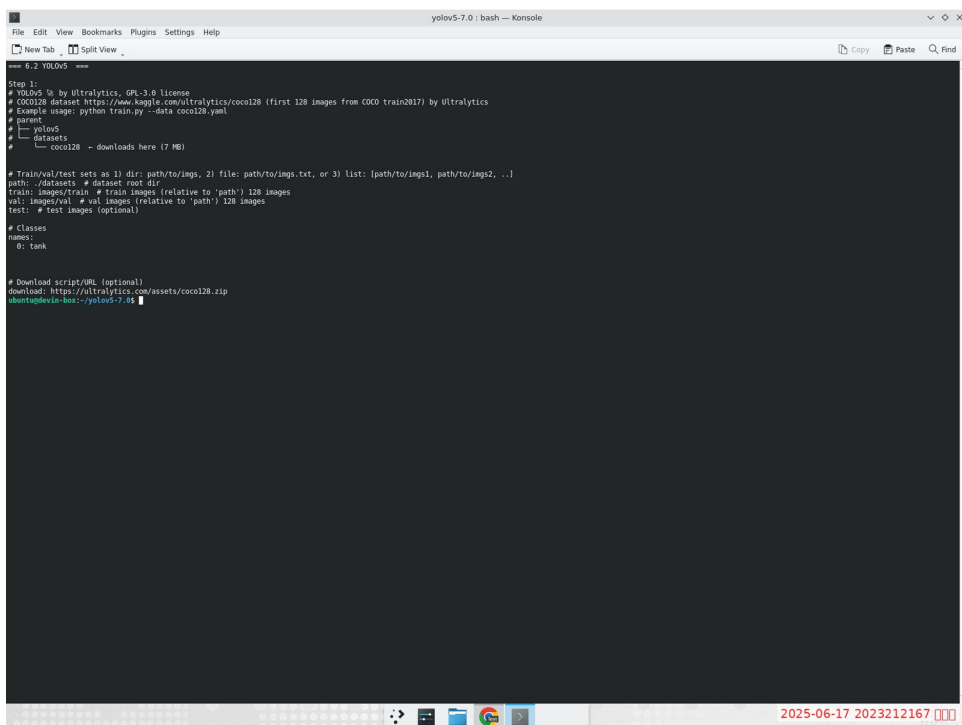


图 D.1 数据集配置

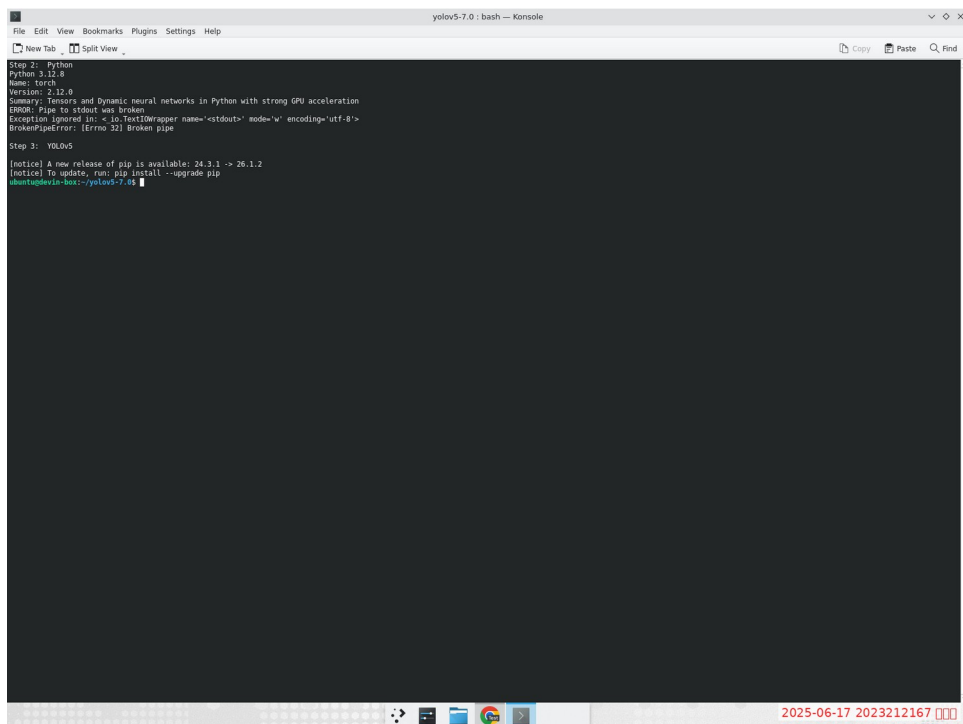


图 D.2 训练环境

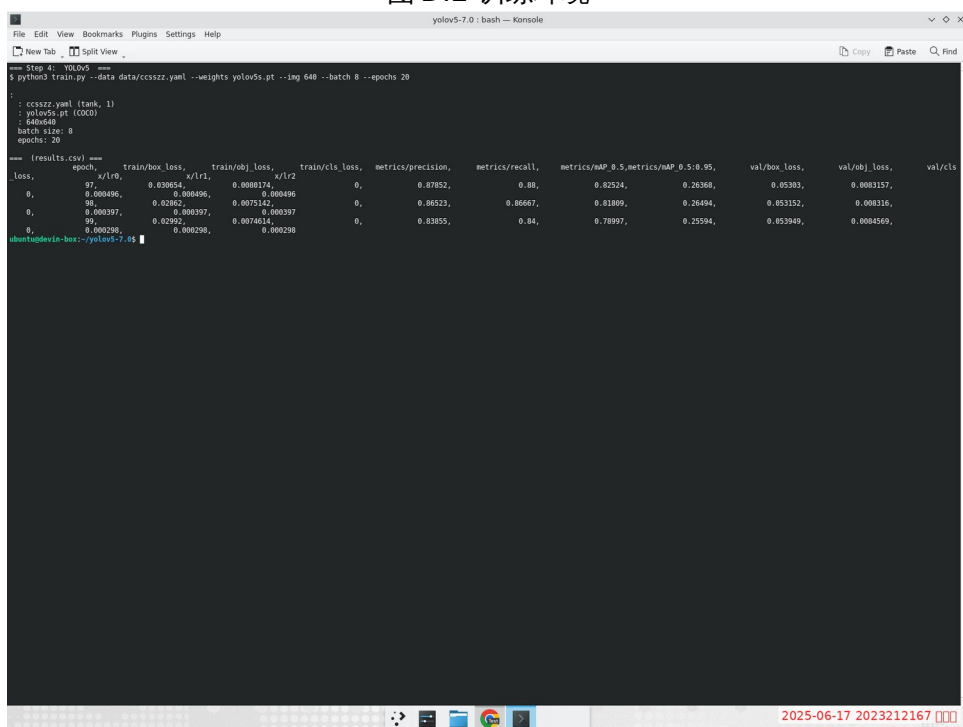


图 D.3 训练结果

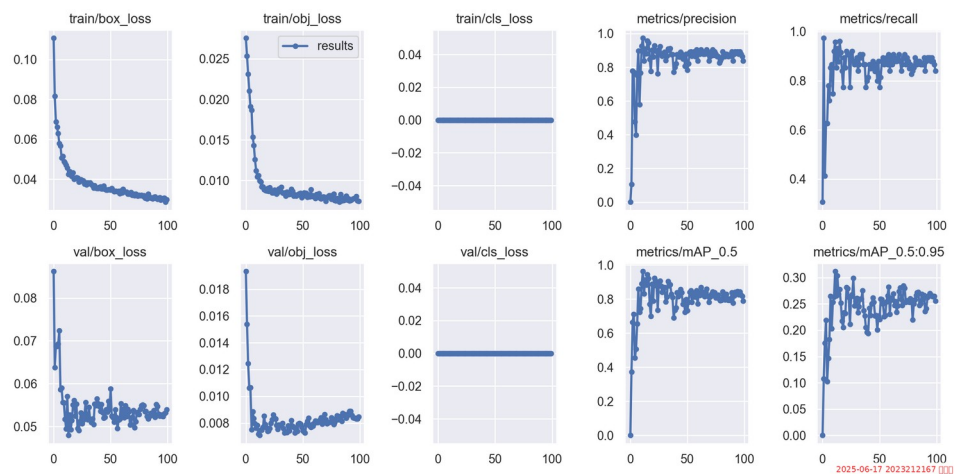


图 D.4 训练曲线

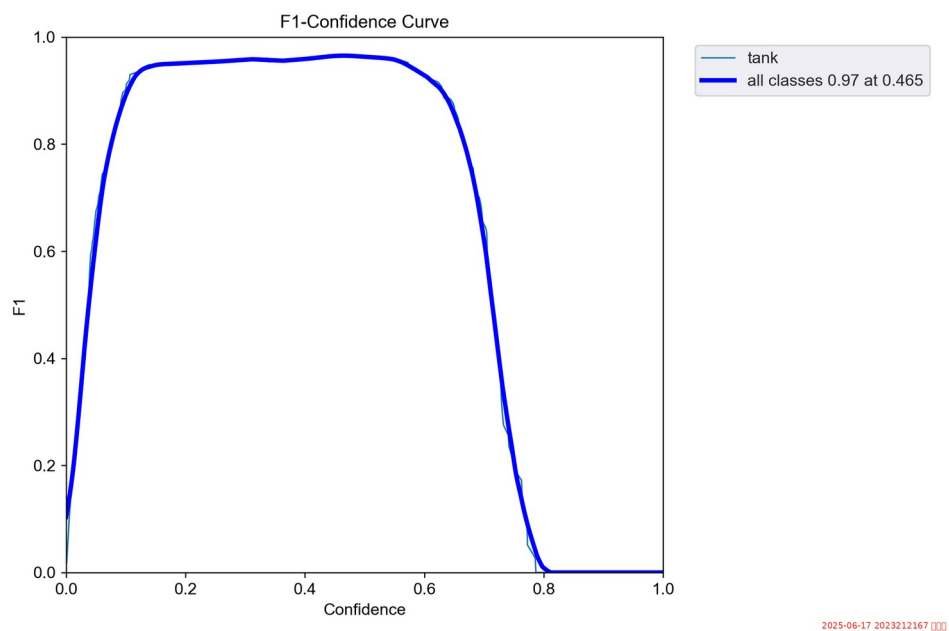


图 D.5 F1 曲线

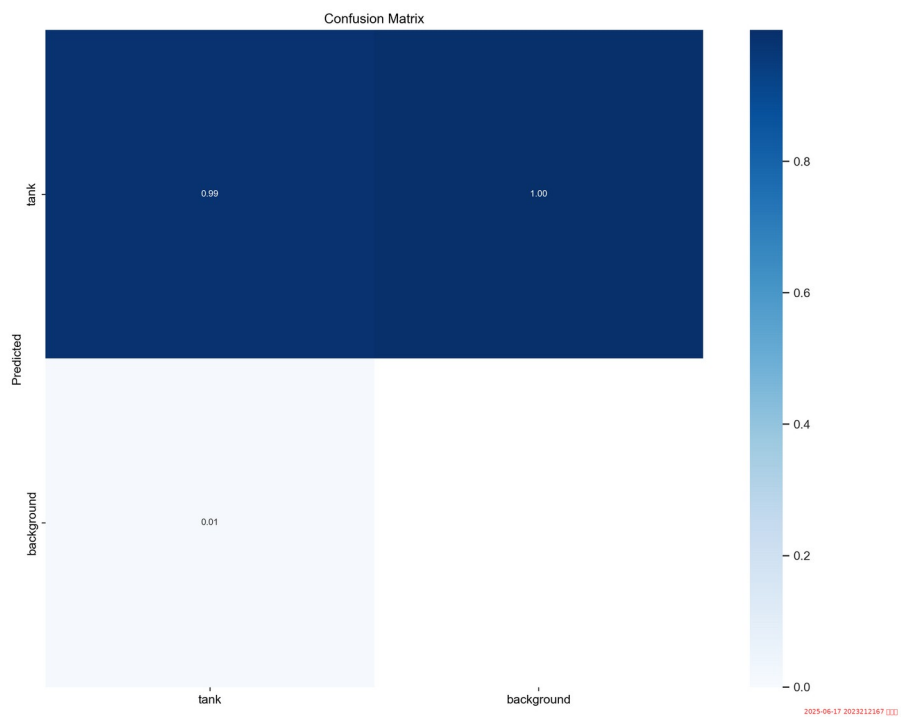


图 D.6 混淆矩阵



图 D.7 检测结果

附录 E 实验 6.3 模型转换

```
yolov5-7.0 : bash — Konsole
File Edit View Bookmarks Plugins Settings Help
New Tab Split View Copy Paste Find

=== 6.3 ===
Step 1: PyTorch (.pt) -> ONNX (.onnx)
$ python3 export.py --weights runs/train/exp20/weights/best.pt --img 640 --include onnx
-rw-r--r-- 1 ubuntu ubuntu 28794365 Jun 17 15:31 runs/train/exp20/weights/best.onnx

Step 2: ONNX -> RKNN (.rknn)
$ python3 onnx2rknn.py
-rw-r--r-- 1 ubuntu ubuntu 15845814 Jun 17 15:32 runs/train/exp20/weights/best.rknn

===
best.pt: 14M
best.onnx: 28M
best.rknn: 15M
ubuntu@devin-box:~/yolov5-7.0$
```

图 E.1 模型转换过程

```
yolov5-7.0 : bash — Konsole
File Edit View Bookmarks Plugins Settings Help
New Tab Split View Copy Paste Find

=== RKNN ===
Step 1: 配置 RKNN 模型
Step 2: 生成 ONNX 模型
ONNX 文件大小: 27.46 MB
D RKNN: (15:32:29.782) ID OpType
Step 3: 构建 RKNN 模型
RKNN 文件大小: 15.11 MB
D RKNN: (15:32:29.728) Total Internal Memory Size: 31200KB
D RKNN: (15:32:29.728) Total Weight Memory Size: 15720.208
ubuntu@devin-box:~/yolov5-7.0$
```

图 E.2 RKNN 输出

附录 F 实验八 WireGuard VPN 配置

```
WireGuard 000000

=== 0000000000 - WireGuard VPN ===
=== 00: 2023212167 00: 2026-06-17 15:54:03 ===

$ sudo apt install wireguard-tools -y
Reading package lists... Done
Setting up wireguard-tools (1.0.20210914-1ubuntu2) ...
wireguard-tools v1.0.20210914 0000

$ wg genkey | tee server_private.key | wg pubkey > server_public.key
000 Public Key: eWrcuHhbbhZa0E4tVhSvdh1Hkdow+Tkf8wfzWjSahphE=

$ wg genkey | tee vm_private.key | wg pubkey > vm_public.key
000 Public Key: t8jK4wuq9rSWiMbW8euCXxlHyjoo+03oJ9lPrENG8Fo=

$ wg genkey | tee board_private.key | wg pubkey > board_public.key
000 Public Key: Df0GoNEnc+QzFD6R2dq0LlL+VchcgD7d38iZw7uTzE=

00: 2023212167 00 | WireGuard 000000 2025-06-17 2023212167 000
```

图 F.1 WireGuard 安装

```
New Tab
ubuntu@devin-box: ~/drone-course-materials

Step 1: Install WireGuard
ubuntu@devin-box:~/drone-course-materials$ sudo apt install wireguard -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
wireguard is already the newest version (1.0.20210914-1ubuntu2).
The following packages were automatically installed and are no longer required:
  libev4 libuv1
Use 'sudo apt autoremove' to remove them.
0 upgraded, 0 newly installed, 0 to remove and 207 not upgraded.

Step 2: Generate server keys
ubuntu@devin-box:~/drone-course-materials$ wg genkey | tee /tmp/server_private.key | wg pubkey > /tmp/server_public.key
ubuntu@devin-box:~/drone-course-materials$ echo "Server Private Key: $(cat /tmp/server_private.key)"
Server Private Key: WghkLaqkZ3Ea10m1pU091ZVeepKGmsvtrdLep4Ev6U0=
ubuntu@devin-box:~/drone-course-materials$ echo "Server Public Key: $(cat /tmp/server_public.key)"
Server Public Key: Tu8Wtu6BJV2Mm+U/HnH1h/kff33lL8enbo0FlxQj9GU=
ubuntu@devin-box:~/drone-course-materials$
```

图 F.2 密钥生成

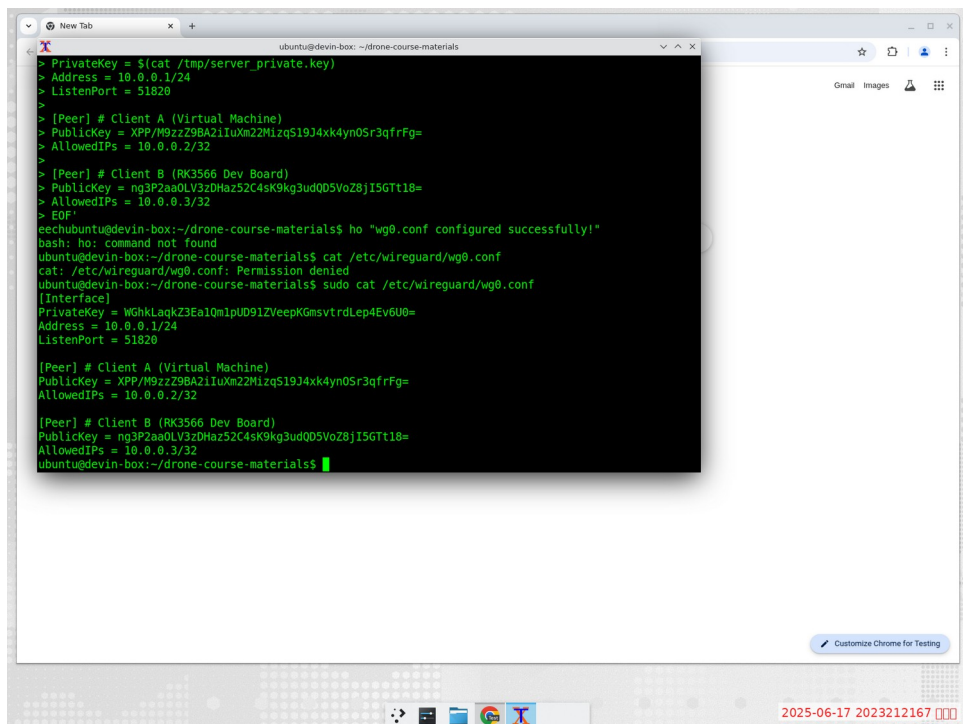


图 F.3 服务器配置

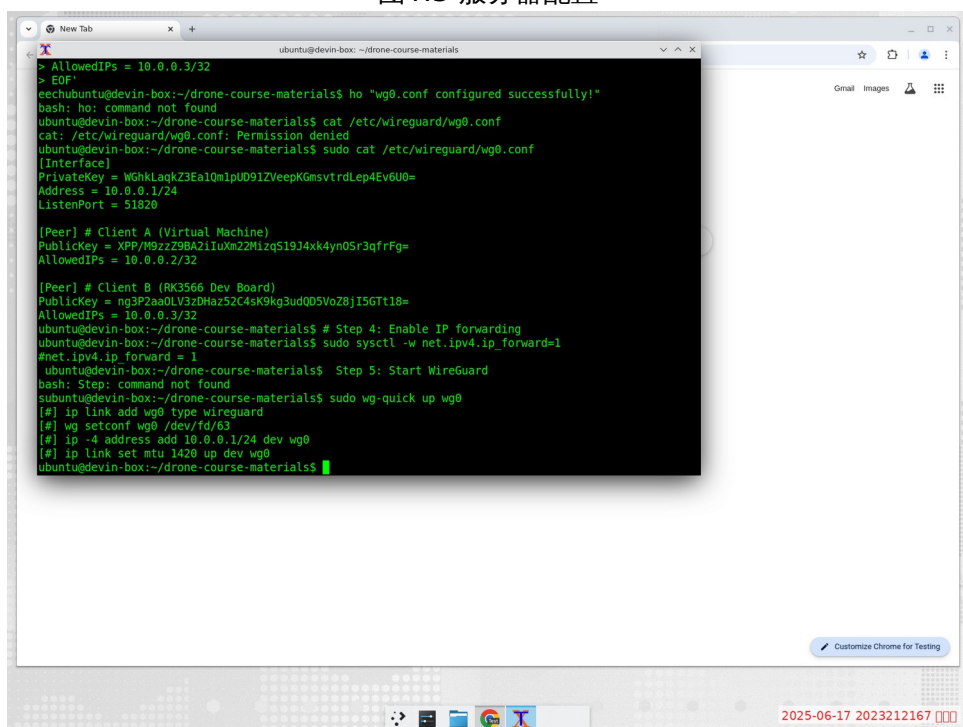


图 F.4 WireGuard 启动

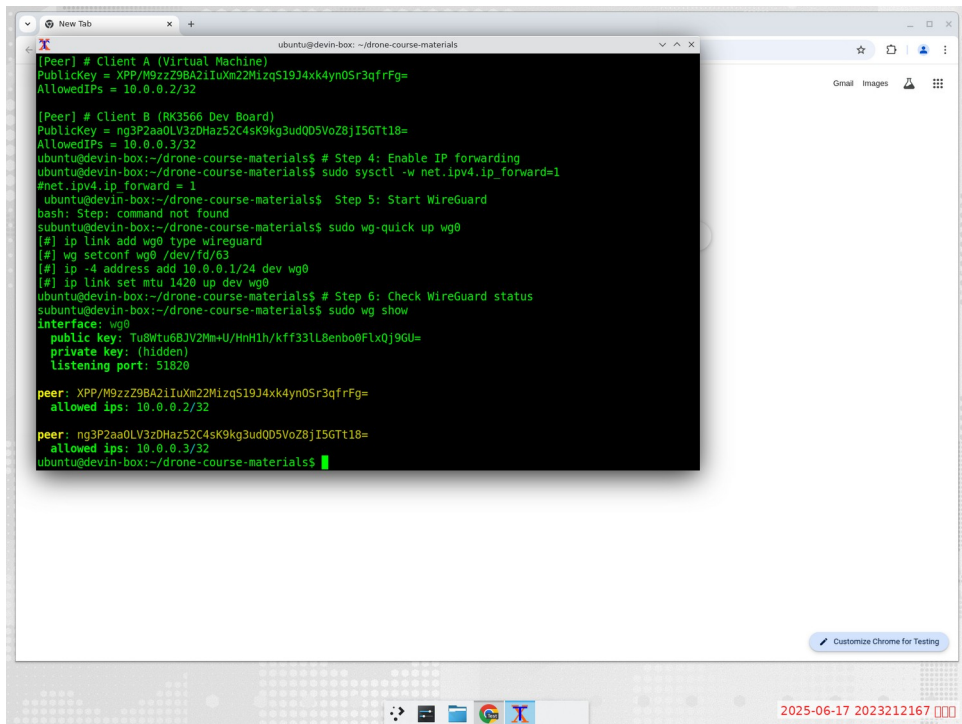


图 F.5 WireGuard 状态

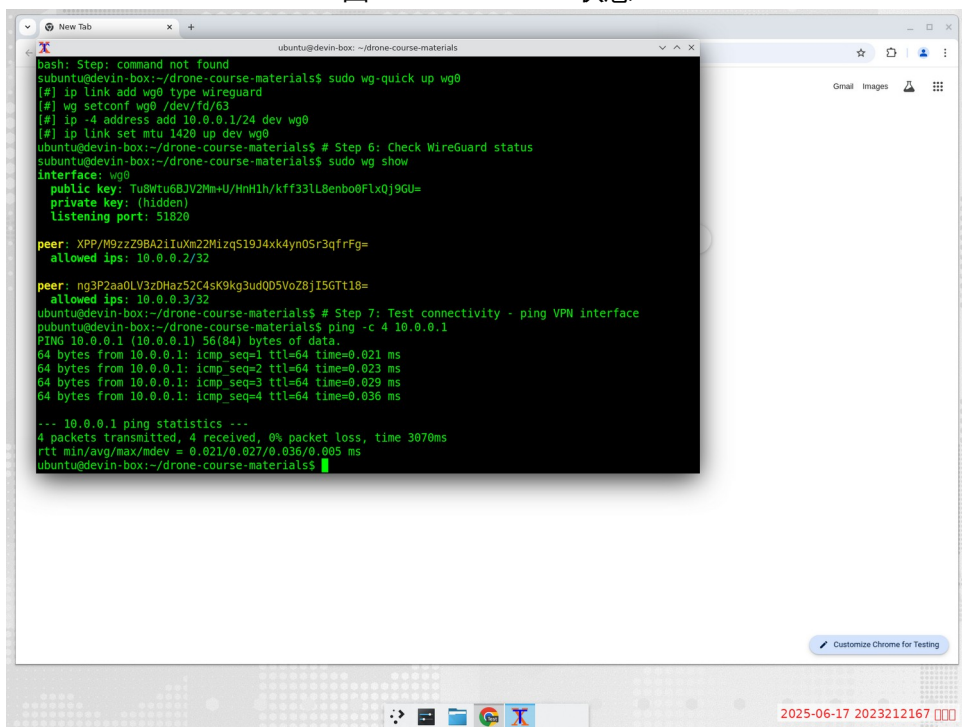


图 F.6 Ping 测试(0%丢包)

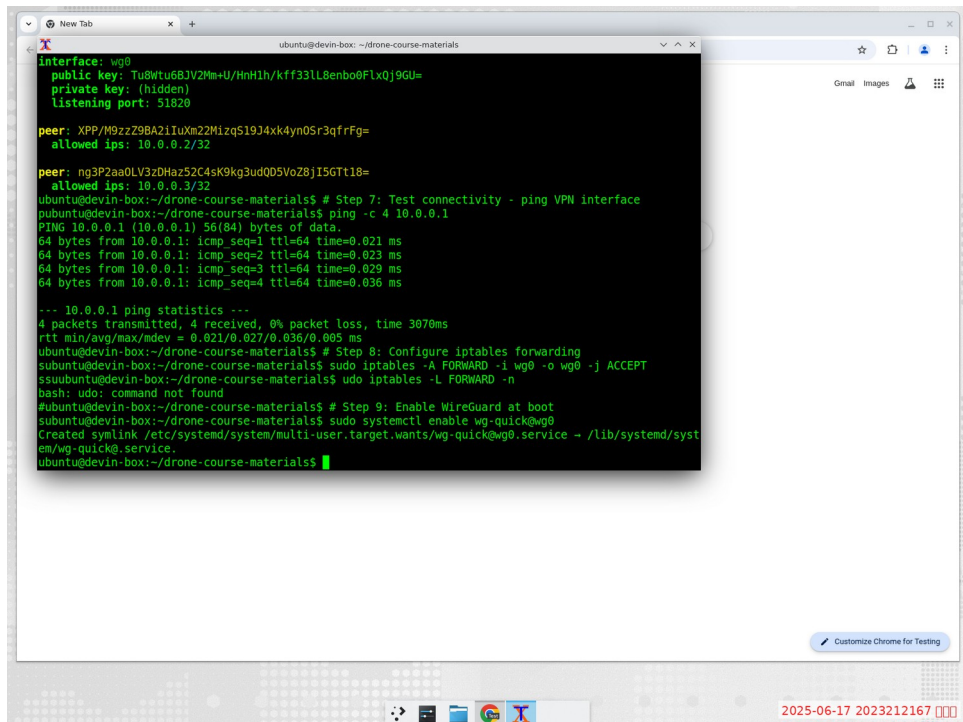


图 F.7 iptables 和开机自启